Content

이번 주차의 학습목표와 학습내용을 확인하세요.

01. 연결된 구조의 이해

배열 구조와 연결된 구조를 비교하여 이해하고 설명할 수 있다.

02. 연결된 구조의 응용

단순연결, 원형연결, 이중연결 구조를 이해하고 설명할 수 있다.

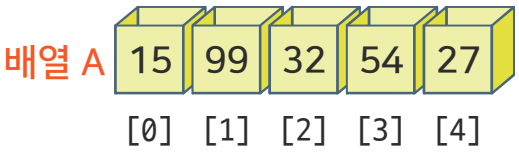
연결된 구조의 이해

연결된 구조의 개념

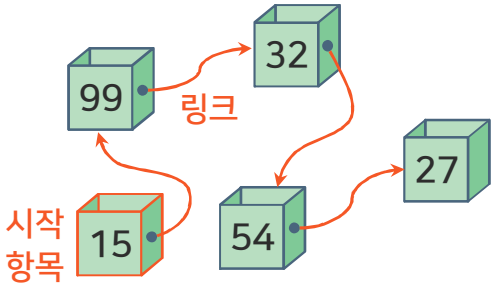
연결된 구조의 특징

1 흩어진 데이터를 링크로 연결해서 관리함

배열 구조의 리스트



연결된 구조의 리스트

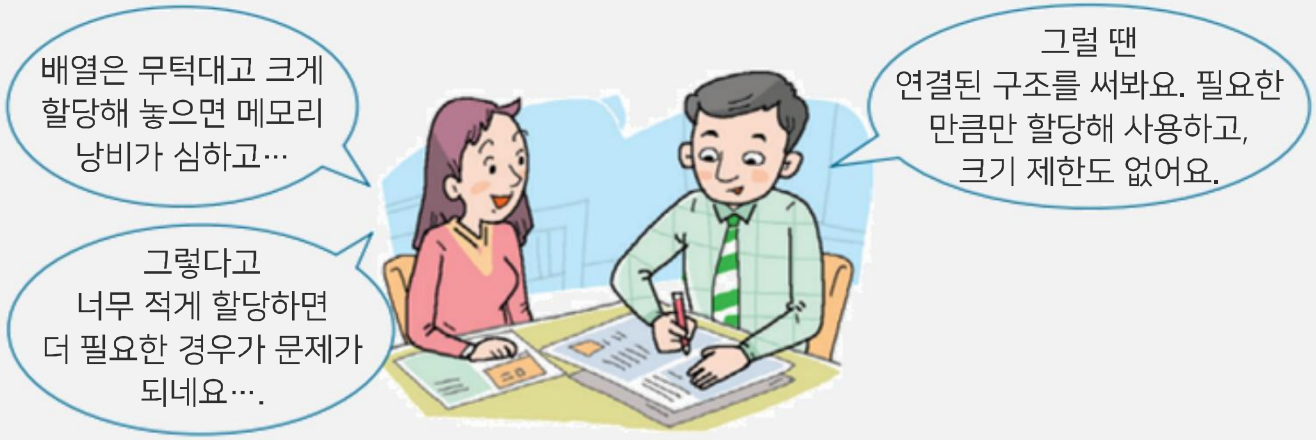


<이미지 출처: 최명규, 천인국. '파이썬으로 쉽게 풀어쓴 자료구조'. 생능출판사>

연결된 구조의 개념

연결된 구조의 특징

2 용량이 고정되어 있지 않음

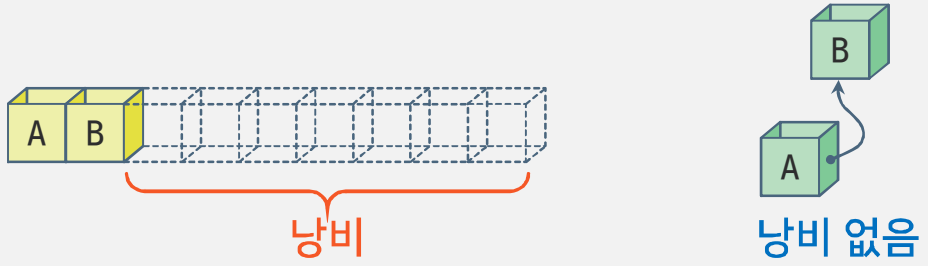


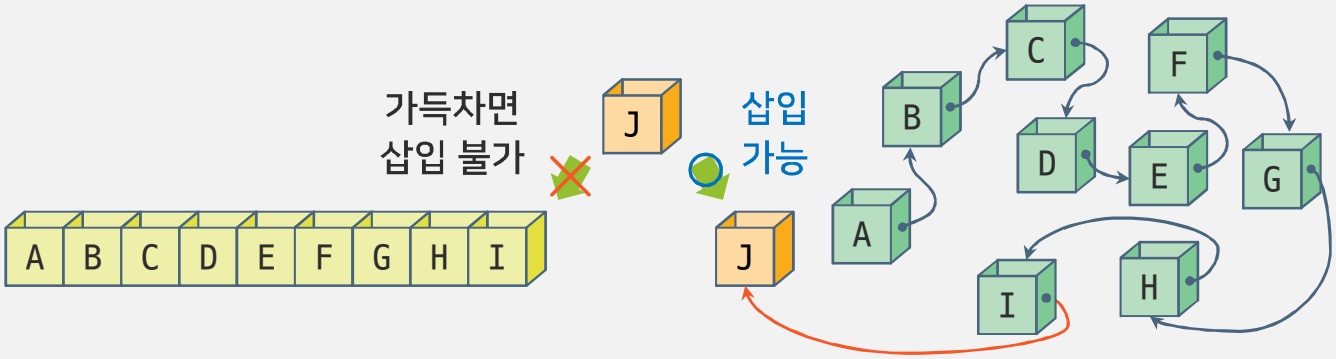
<이미지 출처: 최명규, 천인국. '파이썬으로 쉽게 풀어쓴 자료구조'. 생능출판사>

연결된 구조의 개념

연결된 구조의 특징

2 용량이 고정되어 있지 않음



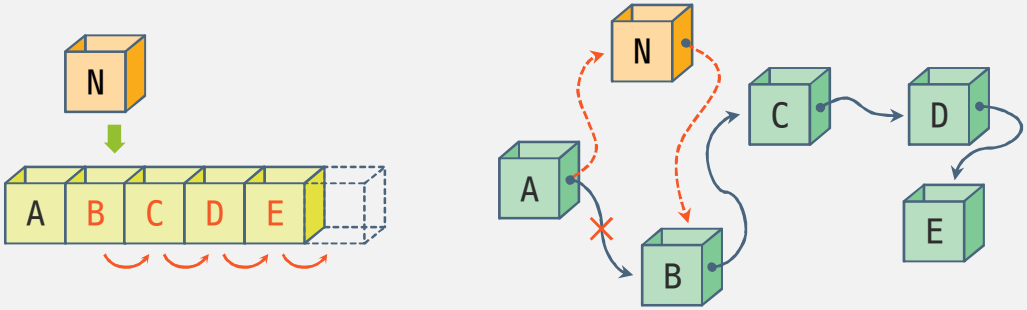


연결된 구조의 개념

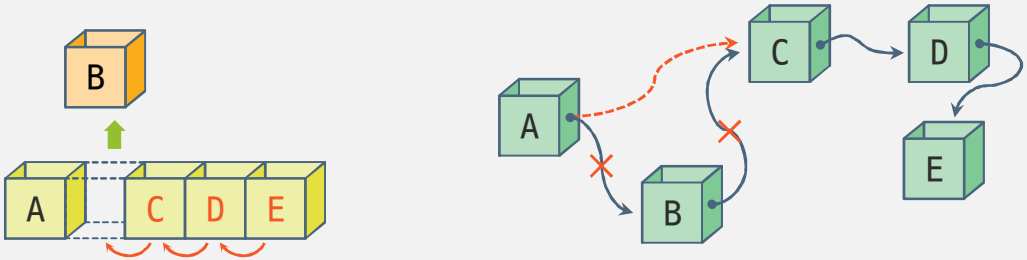
연결된 구조의 특징

3 중간에 자료를 삽입하거나 삭제하기 용이함

항목 N을 B 위치에 삽입할 때



항목 B를 삭제할 때



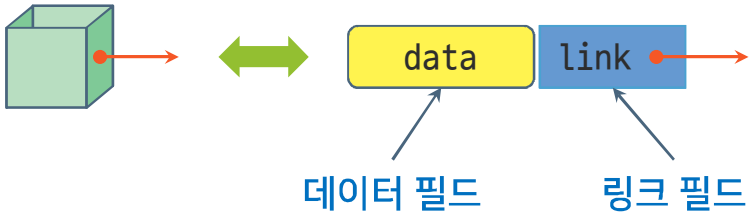
4 n번째 항목에 접근하는데 O(n)의 시간이 소요됨

연결된 구조의 개념

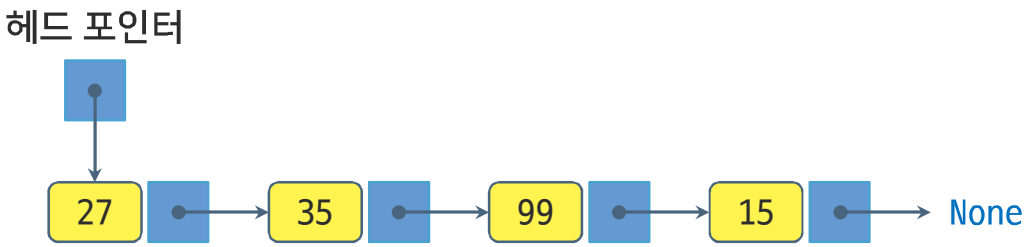
연결 리스트의 구조

노드(node)

- 데이터 필드(data field)
- 하나 이상의 링크 필드(link field)



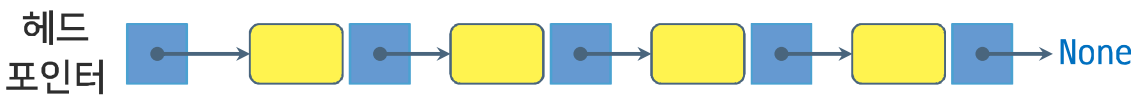
헤드 포인터 (head pointer)



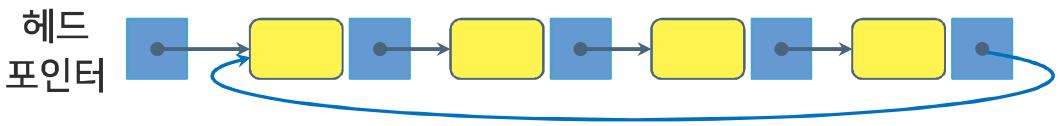
연결된 구조의 개념

연결 리스트의 종류

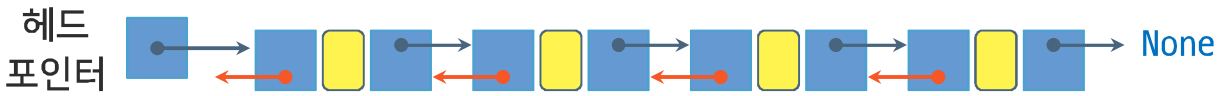
01 단순연결 리스트(singly linked list)



02 원형연결 리스트(circular linked list)



03 이중연결 리스트(doubly linked list)



연결된 구조의 응용

단순연결 리스트

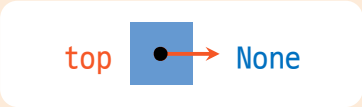
연결된 스택

노드 클래스

```
class Node:                                  # 단순연결 리스트를 위한 노드 클래스
    def __init__(self, elem, link=None):     # 생성자. 디폴트 인수 사용
        self.data = elem                     # 데이터 멤버 생성 및 초기화
        self.link = link                     # 링크 생성 및 초기화
```

연결된 스택 클래스

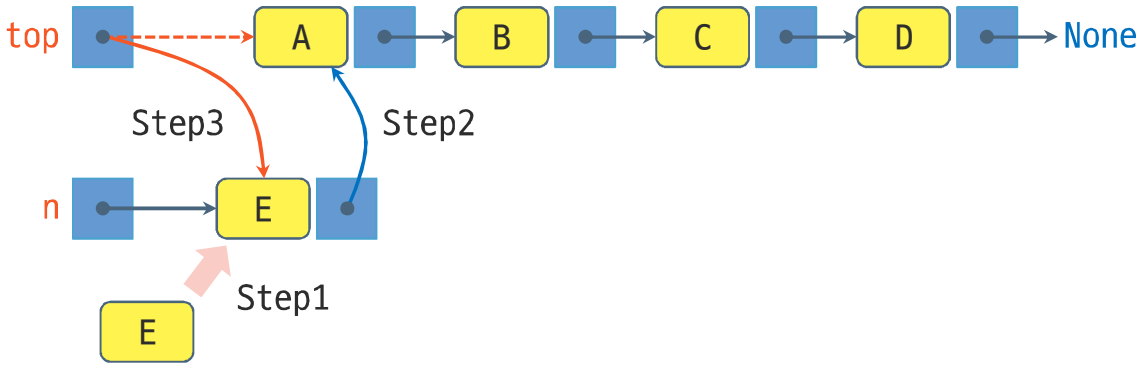
```
class LinkedStack :
    def __init__( self ):        # 생성자
        self.top = None          # top 생성 및 초기화

    def isEmpty( self ): return self.top == None     # 공백상태 검사
    def clear( self ): self.top = None               # 스택 초기화
```



단순연결 리스트

연결된 스택

삽입 연산 | 삭제 연산 | 전체 노드의 방문



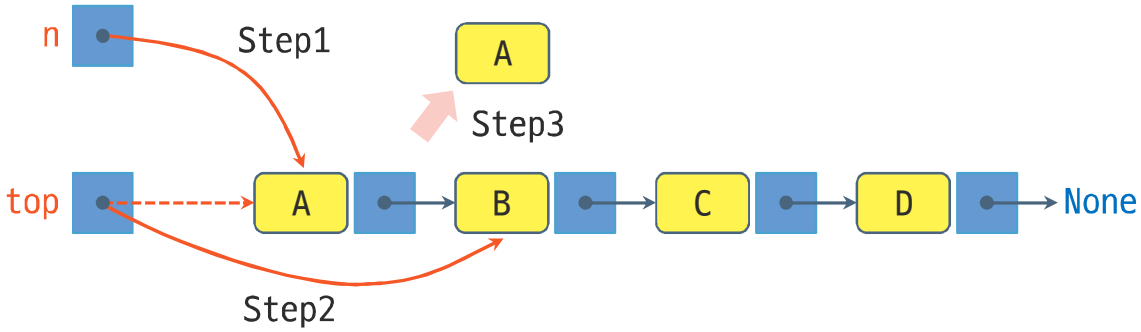

```
def push( self, item ):          # 연결된 스택의 삽입 연산
    n = Node(item, self.top)     # Step1 + Step2
    self.top = n                 # Step3
```

단순연결 리스트

연결된 스택

삽입 연산 | **삭제 연산** | 전체 노드의 방문

- 메모리 해제를 신경 쓸 필요가 없음



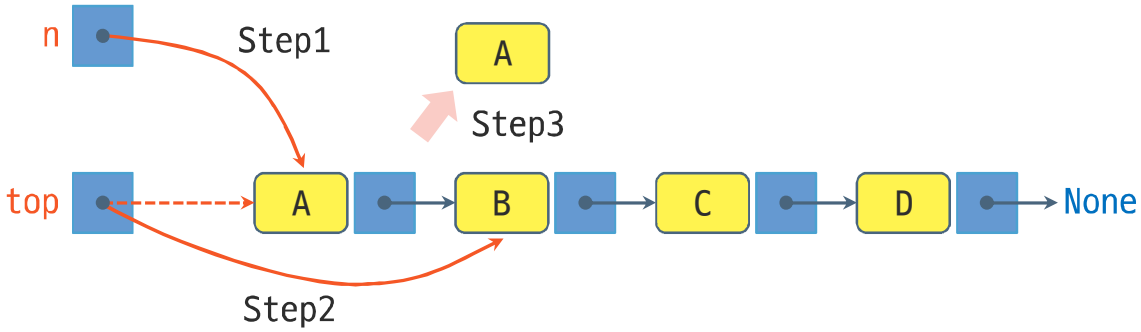
1. 변수 n이 시작 노드를 가리키도록 함: `n = top`
2. top이 다음 노드를 가리키도록 함: `top = n.link`
3. n이 가리키는 노드의 데이터를 반환함: `return n.data`

단순연결 리스트

연결된 스택

삽입 연산 | **삭제 연산** | 전체 노드의 방문

- 메모리 해제를 신경 쓸 필요가 없음

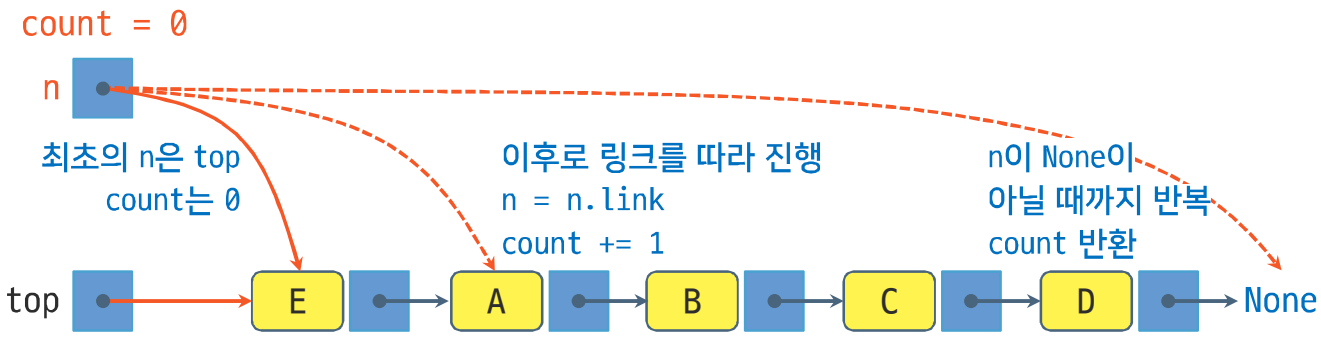



```
def pop(self):                          # 연결된 스택의 삭제 연산
    if not self.isEmpty():              # 공백이 아니면
        n = self.top                    # Step1
        self.top = self.n.link          # Step2
        return n.data                   # Step3
```

단순연결 리스트

연결된 스택

삽입 연산 | 삭제 연산 | **전체 노드의 방문**




```
def size(self):                 # 스택의 항목 수 계산
    node = self.top             # 시작 노드
    count = 0
    while not node == None :    # node가 None이 아닐 때까지
        node = node.link        # 다음 노드로 이동
        count += 1              # count 증가
    return count                # count 반환
```

단순연결 리스트

연결된 리스트

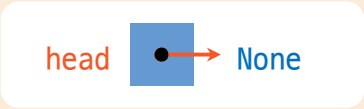
연결된 리스트 구조

단순연결 리스트

연결된 리스트

- **노드 클래스** 연결된 스택에서와 동일
- **연결 리스트 클래스**

```
class LinkedList:                                   # 연결된 리스트 클래스
    def __init__( self ):
        self.head = None

    def isEmpty( self ): return self.head == None          # 공백상태 검사
    def clear( self ): self.head = None                    # 리스트 초기화
    def size( self ): …                             # self.top->self.head로 수정. 코드 동일
    def display(self, msg): …                       # self.top->self.head로 수정. 코드 동일
```



단순연결 리스트

연결된 리스트

메소드

getNode(pos) | getEntry(pos) | replace(pos,elem) | find(val)

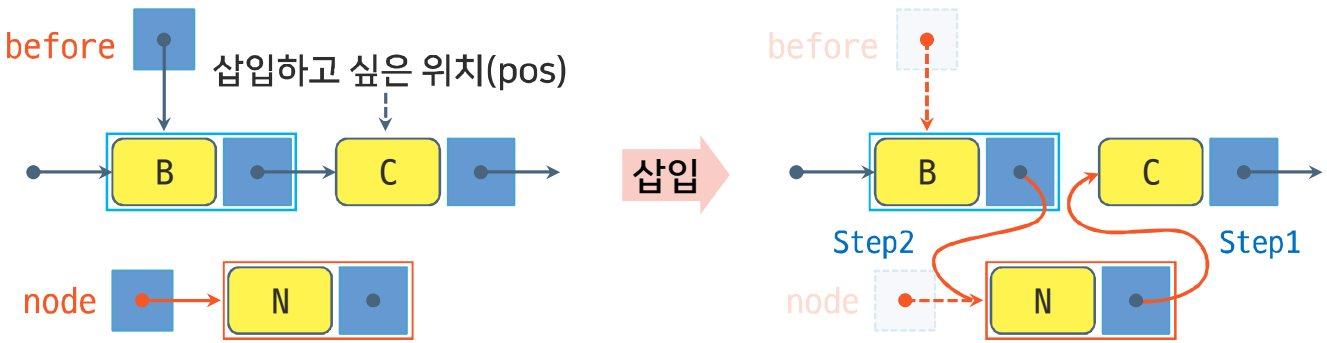
```
def getNode(self, pos) :                    # pos번 째 노드 반환
    if pos< 0: return None
    node = self.head;                       # node는 head부터 시작
    while pos > 0 and node != None :        # pos번 반복
        node = node.link                    # node를 다음 노드로 이동
        pos -= 1                            # 남은 반복 횟수 줄임
    return node                             # 최종 노드 반환
```

```
def getEntry(self, pos):                    # pos번 째 노드의 데이터 반환
    node = self.getNode(pos)                # pos번 째 노드
    if node: == None : return None          # 찾는 노드가 없는 경우
    else : return node.data                 # 그 노드의 데이터 필드 반환
```

단순연결 리스트

연결된 리스트

삽입 연산 | 삭제 연산

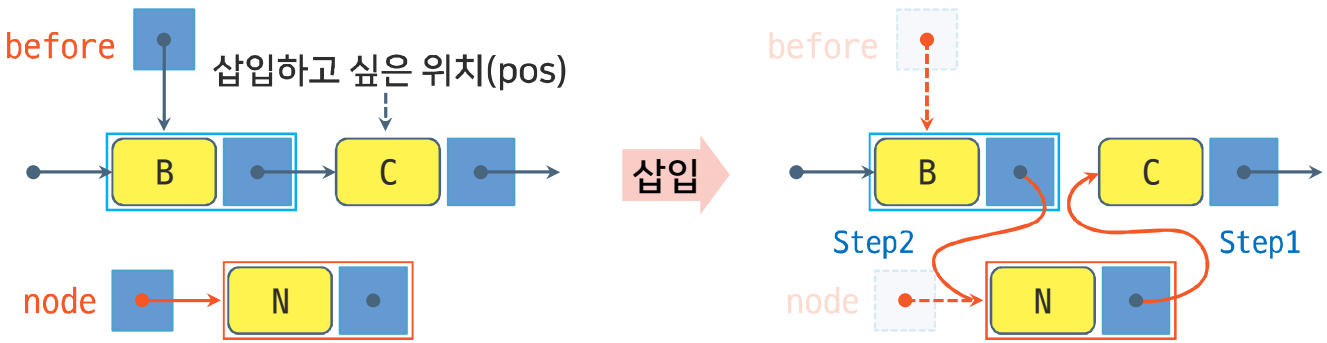


1. 노드 N이 노드 C를 가리키게 함: `node.link = before.link`
2. 노드 B가 노드 N을 가리키게 함: `before.link = node`

단순연결 리스트

연결된 리스트

삽입 연산 | 삭제 연산



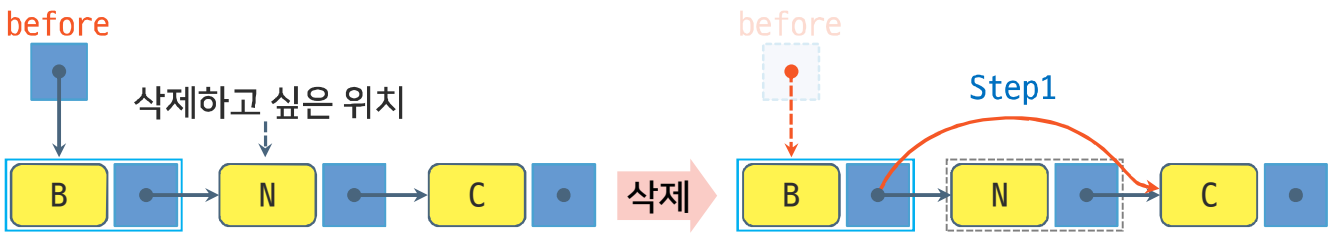

```
def insert(self, pos, elem) :
    before = self.getNode(pos-1)            # before 노드를 찾음
    if before == None :                     # 맨 앞에 삽입하는 경우
        self.head = Node(elem, self.head)   # 맨 앞에 삽입함
    else :                                  # 중간에 앞에 삽입하는 경우
        node = Node(elem, before.link)      # 노드 생성 + Step1
        before.link = node                  # Step2
```

(2/2)

단순연결 리스트

연결된 리스트

삽입 연산 | **삭제 연산**




```
def delete(self, pos) :
    before = self.getNode(pos-1)          # before 노드를 찾음
    if before == None :                   # 시작 노드를 삭제
        if self.head is not None :        # 공백이 아니면
            self.head = self.head.link    # head를 다음으로 이동
    elif before.link != None :            # 중간에 있는 노드 삭제
        before.link = before.link.link    # Step1
```

단순연결 리스트

연결된 리스트

테스트 코드

```
s = LinkedList()
s.display('단순연결리스트로 구현한 리스트(초기상태):')
s.insert(0, 10);          s.insert(0, 20);          s.insert(1, 30)
s.insert(s.size(), 40);   s.insert(2, 50)
s.display("단순연결리스트로 구현한 리스트(삽입x5):")
s.replace(2, 90)
s.display("단순연결리스트로 구현한 리스트(교체x1):")
s.delete(2);     s.delete(s.size() - 1);     s.delete(0)
s.display("단순연결리스트로 구현한 리스트(삭제x3):")
s.clear()
s.display("단순연결리스트로 구현한 리스트(정리후):")
```

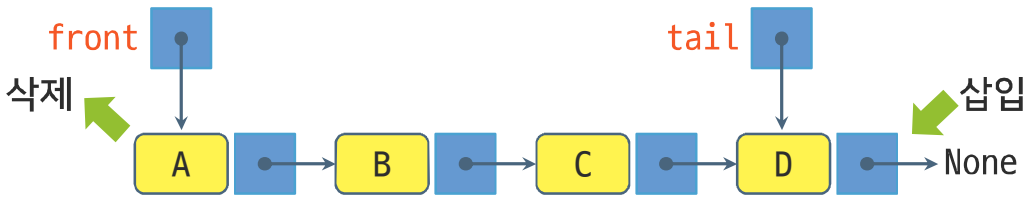
C:\WINDOWS\system32\cmd.exe

```
단순연결리스트로 구현한 리스트(초기상태): None
단순연결리스트로 구현한 리스트(삽입x5): 20->30->50->10->40->None
단순연결리스트로 구현한 리스트(교체x1): 20->30->90->10->40->None
단순연결리스트로 구현한 리스트(삭제x3): 30->10->None
단순연결리스트로 구현한 리스트(정리후): None
```

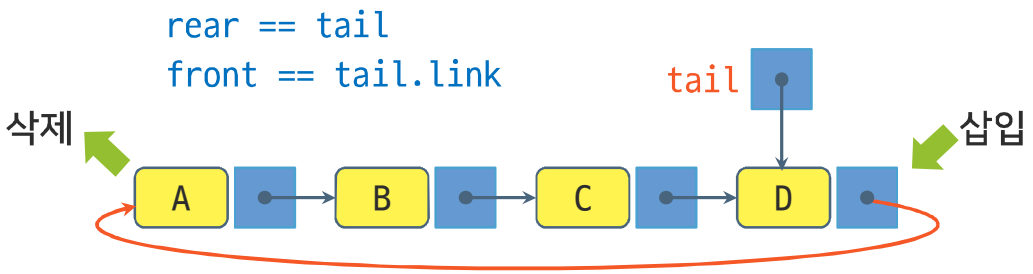
원형연결 리스트

연결된 큐

단순연결 리스트로 구현한 큐



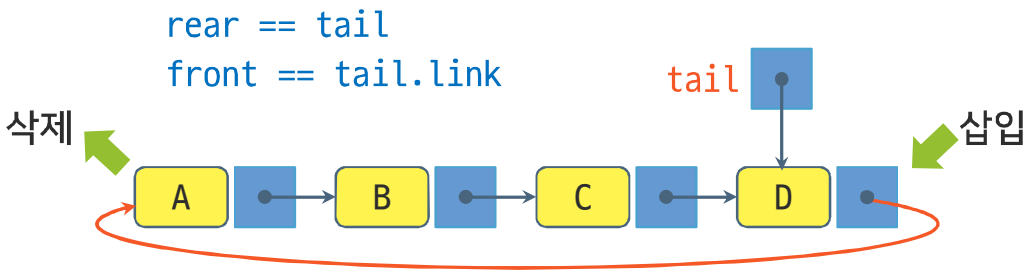
원형연결 리스트로 구현한 큐



원형연결 리스트

연결된 큐

원형연결 리스트로 구현한 큐



- rear의 link가 front 가리킴
  - ➡ 모든 항목들이 연결되어 하나의 포인터로 처리 가능
- tail 포인터를 사용하면 rear와 front에 바로 접근 가능하여 효율적
  - ⬌ front 사용하면 rear에 바로 접근 불가하여 비효율적

원형연결 리스트

연결된 큐

클래스

```
class CircularLinkedQueue:
    def __init__( self ):                          # 생성자 함수
        self.tail = None                           # tail: 유일한 데이터

    def isEmpty( self ): return self.tail == None  # 공백상태 검사
    def clear( self ): self.tail = None            # 큐 초기화
    def peek( self ):                              # peek 연산
        if not self.isEmpty():                     # 공백이 아니면
            return self.tail.link.data             # front의 data를 반환
```

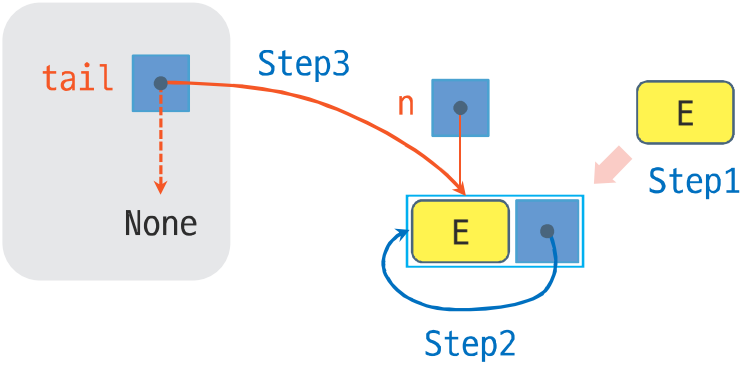
원형연결 리스트

연결된 큐

삽입 연산 | 삭제 연산 | 마지막 노드 검사 | 전체 노드의 방문

```
def enqueue( self, item ):
    node = Node(item, None)
    if self.isEmpty() :
        node.link = node
        self.tail = node
    else :
        node.link = self.tail.link
        self.tail.link = node
        self.tail = node
```

CASE1 큐가 공백상태인 경우의 삽입 연산



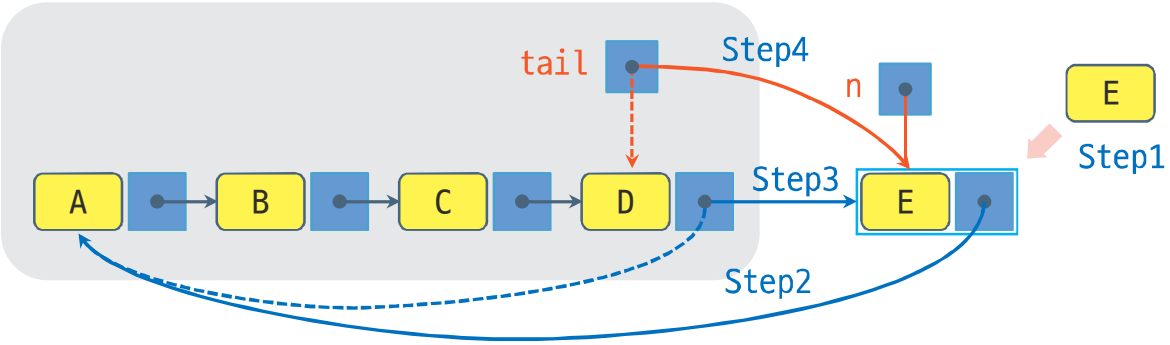
원형연결 리스트

연결된 큐

삽입 연산 | 삭제 연산 | 마지막 노드 검사 | 전체 노드의 방문

```
def enqueue( self, item ):
    node = Node(item, None)
    if self.isEmpty() :
        node.link = node
        self.tail = node
    else :
        node.link = self.tail.link
        self.tail.link = node
        self.tail = node
```

CASE2 큐가 공백상태가 아닌 경우의 삽입 연산



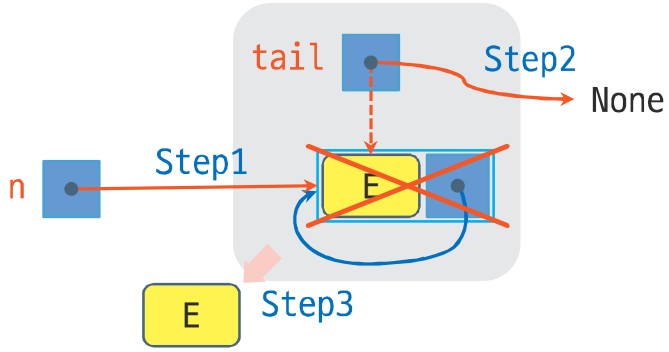
원형연결 리스트

연결된 큐

삽입 연산 | **삭제 연산** | 마지막 노드 검사 | 전체 노드의 방문

```
def dequeue(self):
    if not self.isEmpty():
        data = self.tail.link.data
        if self.tail.link == self.tail :
            self.tail = None
        else:
            self.tail.link = self.tail.link.link
        return data
```

CASE1 큐가 하나의 항목을 갖는 경우의 삭제 연산



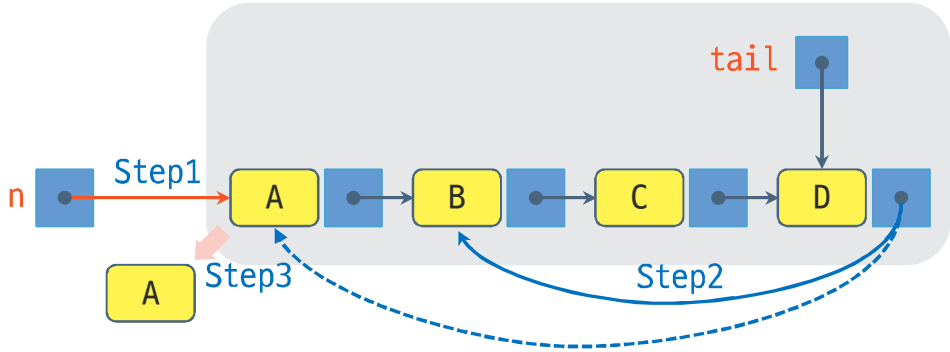
원형연결 리스트

연결된 큐

삽입 연산 | **삭제 연산** | 마지막 노드 검사 | 전체 노드의 방문

```
def dequeue(self):
    if not self.isEmpty():
        data = self.tail.link.data
        if self.tail.link == self.tail :
            self.tail = None
        else:
            self.tail.link = self.tail.link.link
        return data
```

CASE2 큐가 여러 개의 항목을 갖는 경우의 삭제 연산



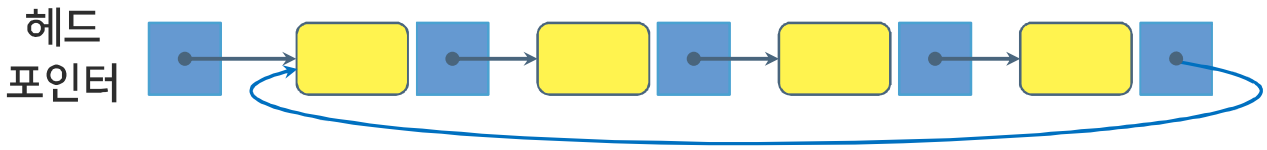
➡ 용량 제한이 없고 삽입/삭제가 모두 O(1)

원형연결 리스트

연결된 큐

삽입 연산 | 삭제 연산 | **마지막 노드 검사** | 전체 노드의 방문

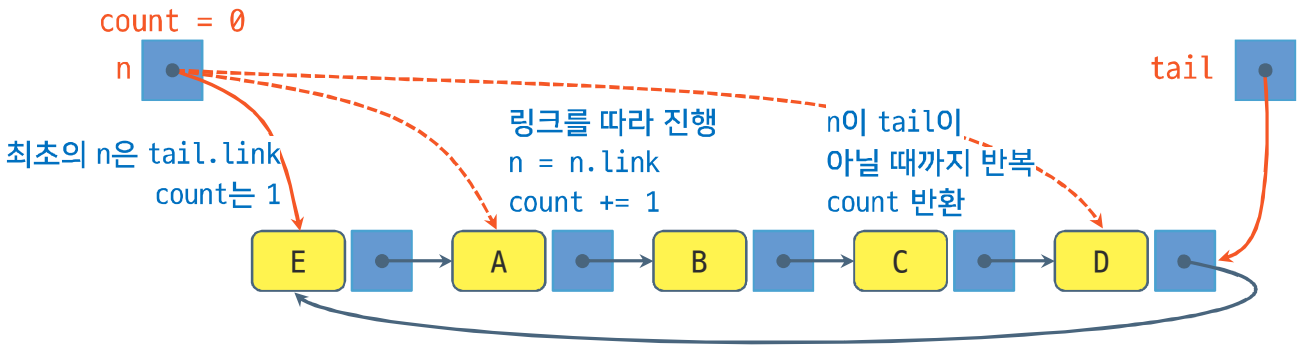
- 마지막 노드의 링크가 None이 아님
- 마지막 노드의 링크가 첫 번째 노드(헤드 포인터)를 가리키는지를 통해 마지막 노드 검사를 할 수 있음
- 마지막 노드 검사 시, 무한 반복되는 알고리즘이 나타날 수 있으므로 구현에 주의



원형연결 리스트

연결된 큐

삽입 연산 | 삭제 연산 | 마지막 노드 검사 | **전체 노드의 방문**




```
def size( self ):
    if self.isEmpty(): return 0          # 공백: 0반환
    else :                               # 공백이 아니면
        count = 1                        # count는 최소 1
        node = self.tail.link            # node는 front부터 출발
        while not node == self.tail:     # node가 rear가 아닌 동안
            node = node.link             # 이동
            count += 1                   # count 증가
         return count                    # 최종 count 반환
```

원형연결 리스트

연결된 큐

테스트 코드

▶ 용량 제한이 없고, 삽입 및 삭제 모두 O(1)

```
q = CircularLinkedQueue()                # 연결된 큐 만들기

for i in range(8): q.enqueue(i)          # 0, 1, … 7 삽입 (f=0, r=8)
q.display()                              # 원형 큐에서 구현한 print()호출
for i in range(5): q.dequeue();          # 5번 삭제 (f=5, r=8)
q.display()
for i in range(8,14): q.enqueue(i)       # 8, 9, … 13 삽입 (f=5, r=4)
q.display
```

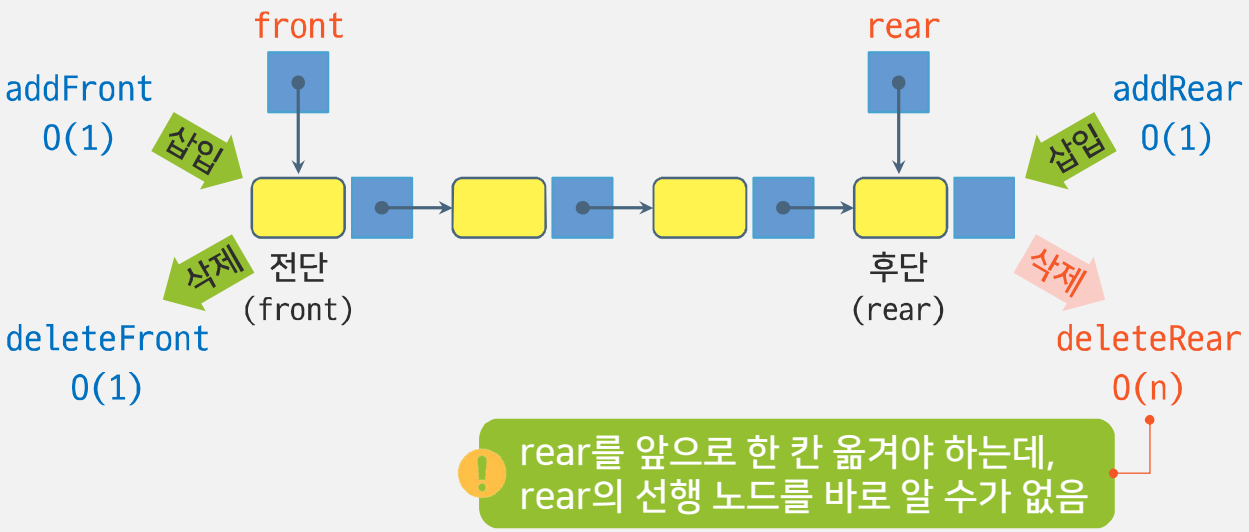
C:\WINDOWS\system32\cmd.exe

```
CircularLinkedQueue: 0 1 2 3 4 5 6 7
CircularLinkedQueue: 5 6 7
CircularLinkedQueue: 5 6 7 8 9 10 11 12 13
```

이중연결 리스트

연결된 덱

단순연결 리스트로 구현한 덱

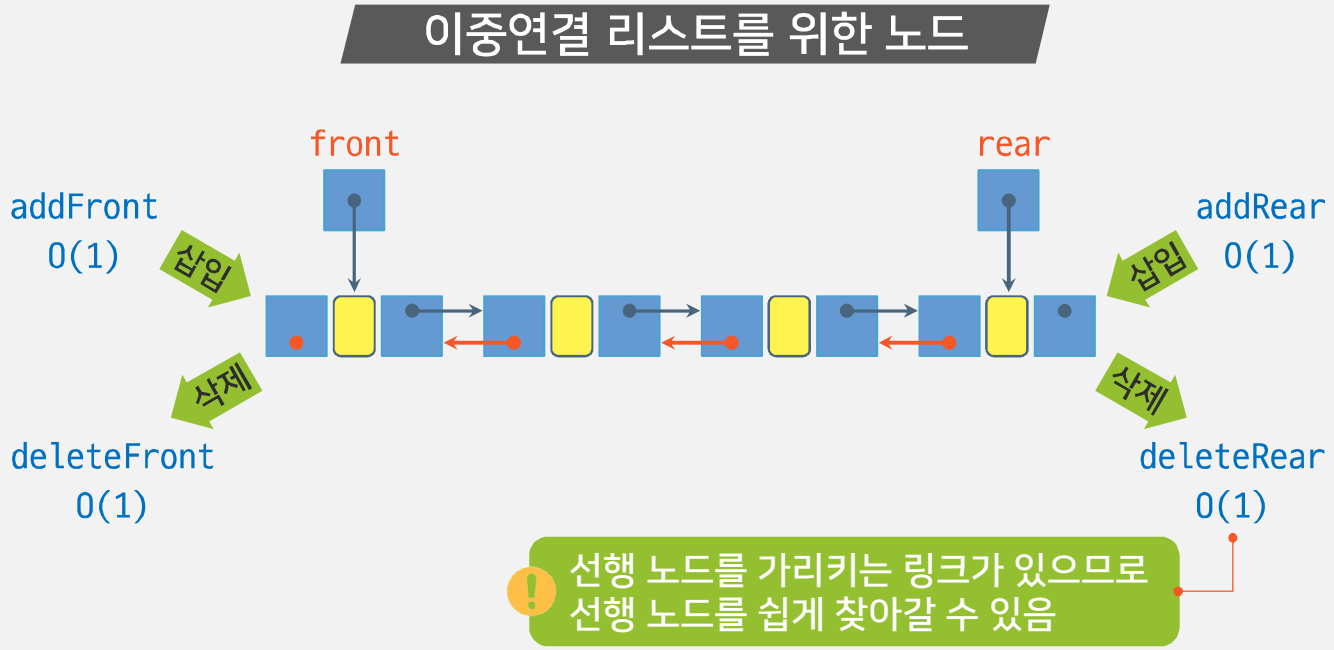


rear를 앞으로 한 칸 옮겨야 하는데,
rear의 선행 노드를 바로 알 수가 없음

이중연결 리스트를 사용하여 해결

이중연결 리스트

연결된 덱



이중연결 구조의 덱에서 deleteRear 연산이 O(1) 소요

이중연결 리스트

연결된 덱

이중연결의 장점

- 선행노드를 O(1)에 바로 탐색 가능

  ❗ 단순연결에서는 선행노드를 시작부터 하나씩 검사하여 O(n) 소요

이중연결의 단점

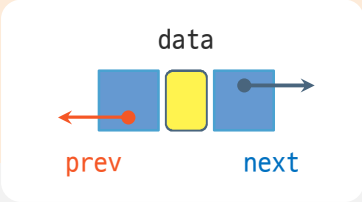
- 모든 연산에서 추가된 링크 관리 필요
- 링크가 많아서 오류 가능성과 메모리 사용 증가

이중연결 리스트

연결된 덱

이중연결 리스트를 위한 노드

```
class DNode:                                # 이중연결 리스트를 위한 노드
    def __init__ (self, elem, prev = None, next = None):
        self.data = elem
        self.prev = prev
        self.next = next
```

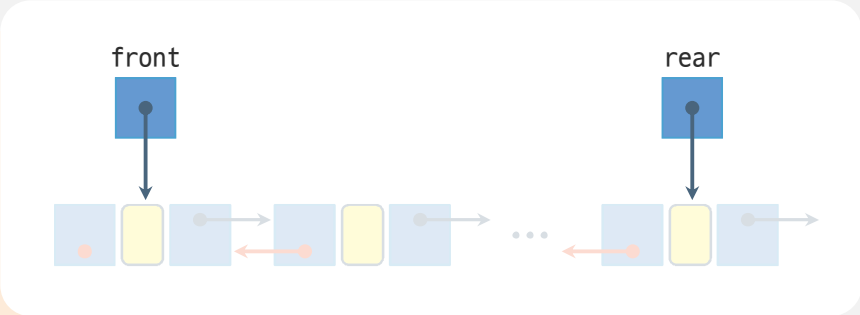


이중연결 리스트

연결된 덱

클래스

```
class DoublyLinkedDeque:
    def __init__( self ):
        self.front = None
        self.rear = None

    def isEmpty( self ): return self.front == None          # 공백상태 검사
    def clear(self): self.front = self.rear = None          # 초기화
    def size(self) : …                # self.top->self.front로 수정. 코드 동일
    def display(self, msg): …         # self.top->self.front로 수정. 코드 동일
```

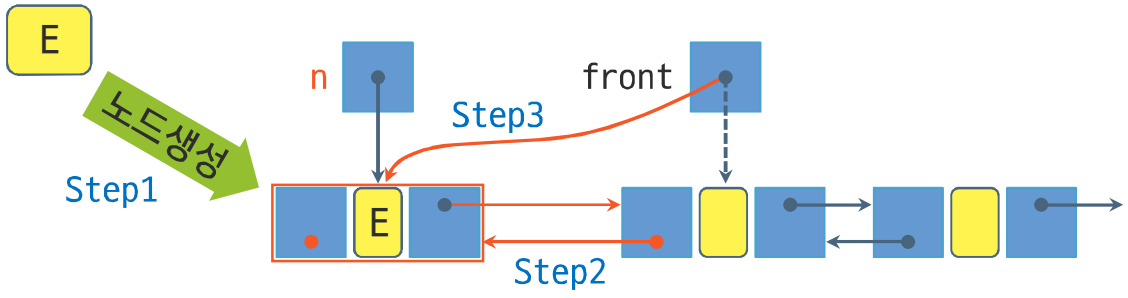


이중연결 리스트

연결된 덱

addFront() | addRear() | deleteFront() | deleteRear()

```
def addFront( self, item ):
    node = DNode(item, None, self.front)        # Step1
    if(self.isEmpty()):                         # 공백이면
        self.front = self.rear = node           # front와 rear가 모두 node
    else :                                      # 공백이 아니면
        self.front.prev = node                  # Step2
        self.front = node                       # Step3
```



(1/2)

이중연결 리스트

연결된 덱

addFront() | **addRear()** | deleteFront() | deleteRear()

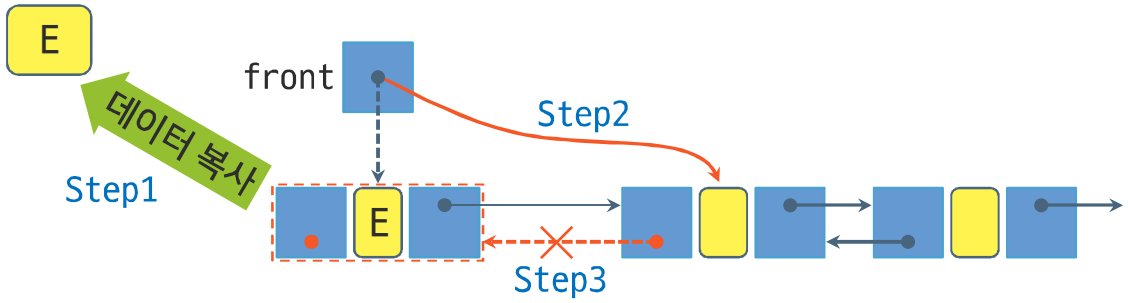
```
def addRear( self, item ):
    node = DNode(item, self.rear, None)        # Step1
    if(self.isEmpty()):                        # 공백이면
        self.front = self.rear = node          # front와 rear가 모두 node
    else :                                     # 공백이 아니면
        self.rear.next = node                  # Step2
        self.rear = node                       # Step3
```

이중연결 리스트

연결된 덱

addFront() | addRear() | **deleteFront()** | deleteRear()

```
def deleteFront( self ):
    if not self.isEmpty():
        data = self.front.data                  # Step1
        self.front = self.front.next            # Step2
        if self.front==None :                   # 노드가 하나 뿐이면
            self.rear None                      # rear도 None으로 설정
        else:
            self.front.prev = None              # Step3
        return data                             # Step4
```



(1/2)

이중연결 리스트

연결된 덱

addFront() | addRear() | deleteFront() | **deleteRear()**

```
def deleteRear( self ):
    if not self.isEmpty():
        data = self.rear.data                    # Step1
        self.rear = self.rear.prev               # Step2
        if self.rear==None:                      # 노드가 하나 뿐이면
            self.front = None                    # front도 None으로 설정
        else:
            self.rear.next = None                # Step3
        return data                              # Step4
```

이중연결 리스트

연결된 덱

테스트 코드

```
dq = DoublyLinkedDeque()                    # 연결된 덱 만들기

for i in range(9):                          # i : 0, 1, 2, … 8
    if i%2==0: dq.addRear(i)                # 짝수는 후단에 삽입
    else: dq.addFront(i)                    # 홀수는 전단에 삽입
dq.display()                                # front=6, rear=5
for i in range(2): dq.deleteFront()         # 전단에서 두 번의 삭제: f=8
for i in range(3): dq.deleteRear()          # 후단에서 세 번의 삭제: r=2
dq.display()
for i in range(9,14): dq.addFront(i)        # i : 9, 10, … 13 : f=3
dq.display()
```

C:\WINDOWS\system32\cmd.exe

```
[f=6, r=5] ==> [7, 5, 3, 1, 0, 2, 4, 6, 8]
[f=8, r=2] ==> [3, 1, 0, 2]
[f=3, r=2] ==> [13, 12, 11, 10, 9, 3, 1, 0, 2]
```

학습평가

학습한 내용을 바탕으로 다음 문제를 풀어보세요.

번호	문제	보기	
1	삽입 또는 삭제 연산이 자주 발생하는 경우 가장 많은 실행시간이 소요되는 자료구조는 무엇인가?	① 단순연결 리스트 ② 배열로 구현된 리스트 ③ 원형연결 리스트 ④ 이중연결 리스트	
2	연결된 구조로 구현된 리스트의 장점으로 가장 옳지 않은 것은?	① 항목 접근의 시간 복잡도가 O(n) ② 용량이 고정되어 있지 않음 ③ 중간에 자료 삽입이 효율적 ④ 중간의 자료 삭제가 효율적	
3	단순 연결리스트를 이용하여 구현한 연결된 스택에 대한 설명 중 가장 옳은 것은?	① 노드 삭제 시, 메모리 해제에 대한 코드가 필요. ② 노드 삽입 시, 포화상태에 대한 검사가 필요 ③ top은 연결된 스택의 시작 노드(가장 먼저 입력된 노드)를 가리킴 ④ 포화상태가 존재하지 않음	
4	None(또는 NULL) 포인터가 존재하지 않는 자료구조는 무엇인가 ?	① 단순연결 리스트 ② 원형연결 리스트 ③ 단순연결 구조의 연결된 스택 ④ 이중연결 리스트	

학습평가

학습한 내용을 바탕으로 다음 문제를 풀어보세요.

번호	문제	보기	
5	원형연결 리스트의 마지막 노드의 특징이 아닌 것은?	① 마지막 노드의 링크가 None이 아님 ② 마지막 노드의 링크가 첫 번째 헤드를 가리킴 ③ 선행 노드를 O(1)에 바로 찾아갈 수 있음 ④ 마직막 노드 검사 시, 무한 반복되는 알고리즘이 나타날 수 있으므로 구현에 주의	
6	단순연결 구조로 덱을 구현하는 경우 후단 삭제 연산을 O(1)에 구현할 수 없는 이유로 옳은 것은?	① 머리 노드를 O(1)에 알 수 없음 ② 꼬리 노드의 이전 노드를 O(1)에 알 수 없음 ③ 꼬리 노드를 O(1)에 알 수 없음 ④ 꼬리 노드의 후속 노드를 O(1)에 알 수 없음	
7	이중연결 구조로 구현된 리스트에 대한 설명으로 옳지 않은 것은?	① 단순연결 구조에 비해 메모리 사용이 더 효율적 ② 두 개의 링크를 가진 노드를 사용 ③ 후속 노드를 O(1)에 바로 탐색 가능 ④ 선행 노드를 O(1)에 바로 탐색 가능	

학습정리

연결된 구조

연결된 구조의 이해

- 연결된 구조는 용량이 고정되어 있지 않고, 중간에 자료를 삽입/삭제하는 과정이 효율적이며, 구현이 복잡하고 n번째 항목에 접근하는데 O(n)이 소요됨
- 노드는 연결된 구조에서 하나의 상자를 의미하며, 데이터 필드와 링크 필드로 구성됨
- 단순연결 노드는 링크 필드가 하나이고, 이중연결 노드는 링크 필드가 두개임

연결된 구조의 응용

- 연결된 스택에서 데이터 멤버 top은 최근 삽입된 노드를 가리키는 포인터임
- 연결된 리스트에서 데이터 멤버 head는 인데스 0를 가리키는 포인터임
- 원형연결 리스트에서 마지막 노드 rear의 링크는 첫 번째 노드 front를 가리킴
- 원형연결은 모든 항목들이 연결되어 하나의 포인터로 처리 가능함
- 이중연결 리스트는 이전 노드와 다음 노드를 가리키는 2개의 링크가 필요함
- 이중연결은 선행 노드를 O(1)로 찾아갈 수 있으나, 링크가 많아 메모리 사용이 증가함
- 이중연결 덱은 단순연결 덱과 달리 모든 삽입 및 삭제 연산이 O(1)에 수행됨